

Graphs and Greedy Algorithm

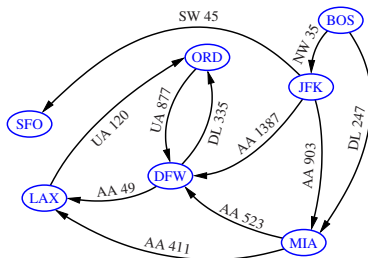
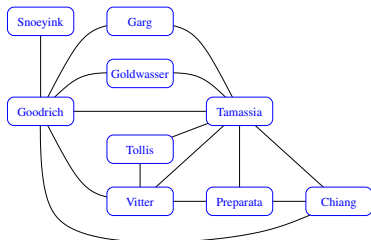
Jianguo Lu

University of Windsor

March 19, 2025

Graph

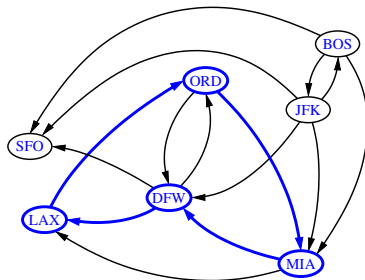
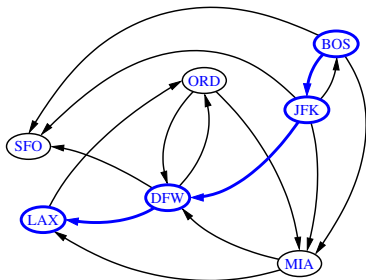
- ▶ A graph is a pair (V, E) , where
 - ▶ V is a set of nodes, called vertices
 - ▶ E is a collection of pairs of vertices, called edges
- ▶ Vertices and edges are positions and store elements
- ▶ Example:
 - ▶ A vertex represents an airport and stores the three-letter airport code
 - ▶ An edge represents a flight route between two airports and stores the mileage of the route



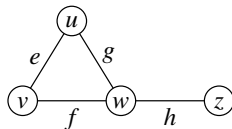
co-author network and flight network

Paths

- ▶ Path: a sequence of edges
- ▶ Directed path: edges are directed and traversed along the directions
 - ▶ $BOS \rightarrow JFK \rightarrow DFW$: directed simple path
- ▶ Cycle: a path that starts and ends at the same vertex
 - ▶ $LAX \rightarrow ORD \rightarrow MIA \rightarrow DFW \rightarrow LAX$: directed simple cycle



Properties for undirected graphs



Suppose that m is the number of edges, and n is the number of nodes.

Property 1 (Edge and degree)

Let $\deg(v)$ denote the degree of v .

$$\sum_{v \in V} \deg(v) = 2m \quad (1)$$

Proof Hint: each edge is counted twice

Property 2 (Edge and nodes)

For an undirected graph with no self-loops and no multiple edges

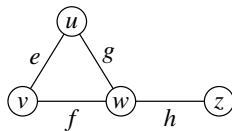
$$m \leq n(n-1)/2 \quad (2)$$

Proof Hint: each vertex has degree at most $(n-1)$.

Complete graph

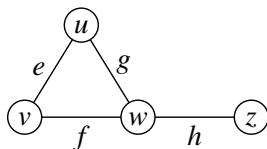
Complete graph is a graph where every pair of nodes is connected.

Strongly Connected Components (SCC)



- ▶ Node u reaches v : there is a directed path from u to v
- ▶ Connected graph: for any two vertices, there is a path between them
- ▶ Strongly connected: for any two vertices u and v , u reaches v and v reaches u .
- ▶ Strongly connected component(SCC): a subgraph that is strongly connected
- ▶ Weakly Connected Component (WCC): a component that, when direction is ignored, every pair of nodes are connected.

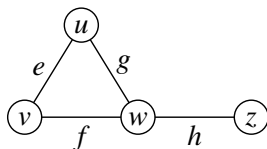
Graph data structures: Matrix (a bad one)



		0	1	2	3
$u \longrightarrow$	0		e	g	
$v \longrightarrow$	1	e		f	
$w \longrightarrow$	2	g	f		h
$z \longrightarrow$	3			h	

- An intuitive way to represent a graph is matrix

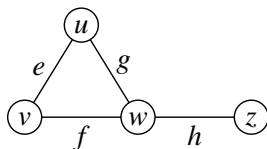
Graph data structures: Matrix (a bad one)



	0	1	2	3
$u \longrightarrow$	0	e	g	
$v \longrightarrow$	1	e	f	
$w \longrightarrow$	2	g	f	h
$z \longrightarrow$	3		h	

- ▶ An intuitive way to represent a graph is matrix
- ▶ Problem with matrix representation?

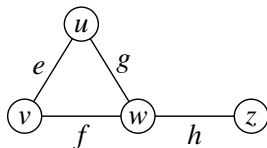
Graph data structures: Matrix (a bad one)



		0	1	2	3
$u \longrightarrow$	0		e	g	
$v \longrightarrow$	1	e		f	
$w \longrightarrow$	2	g	f		h
$z \longrightarrow$	3			h	

- ▶ An intuitive way to represent a graph is matrix
- ▶ Problem with matrix representation?
- ▶ Too many empty cells.
- ▶ The matrix is sparse

Graph data structures: Matrix (a bad one)

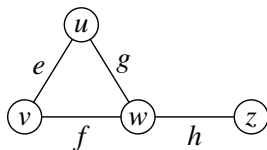


		0	1	2	3
$u \longrightarrow$	0		e	g	
$v \longrightarrow$	1	e		f	
$w \longrightarrow$	2	g	f		h
$z \longrightarrow$	3			h	

- ▶ An intuitive way to represent a graph is matrix
- ▶ Problem with matrix representation?
- ▶ Too many empty cells.
- ▶ The matrix is sparse
 - ▶ e.g., if there are 1 billion (10^9) nodes, on average each node has 10 edges
 - ▶ There are 10^{18} cells, $10^{18} - 10^{10} \approx 10^{18}$ are empty
 - ▶ On average for each useful cell, about 10^8 empty cells are wasted.

Graph data structures: Edge list and Adjacency list

- ▶ Graph is represented by a list of edges.

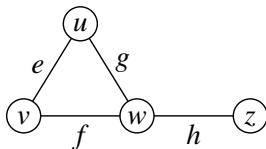


		0	1	2	3
$u \longrightarrow$	0		<i>e</i>	<i>g</i>	
$v \longrightarrow$	1	<i>e</i>		<i>f</i>	
$w \longrightarrow$	2	<i>g</i>	<i>f</i>		<i>h</i>
$z \longrightarrow$	3			<i>h</i>	

Edge list		
u	v	
u	w	
v	u	
v	w	
w	u	
w	v	
w	z	
z	w	

Graph data structures: Edge list and Adjacency list

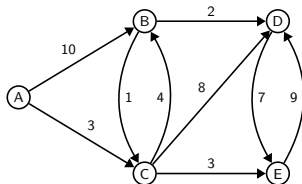
- ▶ Graph is represented by a list of edges.
- ▶ Can be more efficient if we combine multiple neighbours into one linked list



	0	1	2	3
$u \longrightarrow$	0	e	g	
$v \longrightarrow$	1	e	f	
$w \longrightarrow$	2	g	f	h
$z \longrightarrow$	3		h	

Edge list		Adjacency list	
u	v		
u	w		
v	u	u	(v, w)
v	w	v	(u, w)
w	u	w	(u, v, z)
w	v	z	(w)
w	z		
z	w		

Graph implementation using edge list



actual text representation

```
5
9
0 1 10
0 2 3
1 2 1
1 3 2
2 1 4
2 3 8
2 4 3
3 4 7
4 3 9
```

- ▶ Line one: number of nodes
- ▶ Line two: number of edges
- ▶ Each line afterwards: edge (from, to, weight)
- ▶ Use positive integers to denote nodes

Graph class

The constructor reads from a text file that keeps edges of a graph (and the weights in this example)

```
public class Graph {
    public int V;
    public int E;
    //Set of edges for each node
    public Set<Edge>[] edgesOf;
    public int[] indegree;

    public Graph(Scanner in) throws Exception {
        V = in.nextInt();
        E = in.nextInt();
        indegree = new int[V];
        edgesOf = (Set<Edge>[]) new HashSet[V];
        for (int v = 0; v < V; v++)
            edgesOf[v] = new HashSet<Edge>();
        for (int i = 0; i < E; i++) {
            int v = in.nextInt();
            int w = in.nextInt();
            double weight = in.nextDouble();
            edgesOf[v].add(new Edge(v,w,weight));
            indegree[w]++;
        }
    }
}
```

```
public class Edge {
    private final int v;
    private final int w;
    private final double weight;

    public Edge(int v, int w, double weight) {
        this.v = v;
        this.w = w;
        this.weight = weight;
    }

    public int from() {
        return v;
    }

    public int to() {
        return w;
    }

    public double weight() {
        return weight;
    }

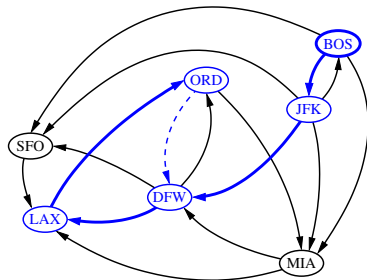
    public String toString() {
        return "(" + v + " " + String.format("%5.2f", weight) + " " + w + ")";
    }
}
```

Graph traversals

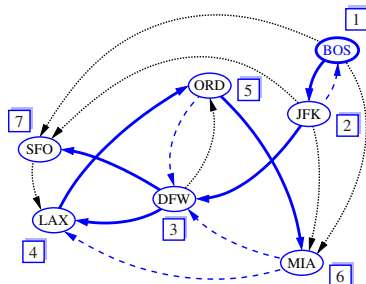
- ▶ Traversal: a systematic procedure for exploring a graph by examining all its vertices
- ▶ Two typical approaches:
 - ▶ DFS
 - ▶ BFS

Depth First Search

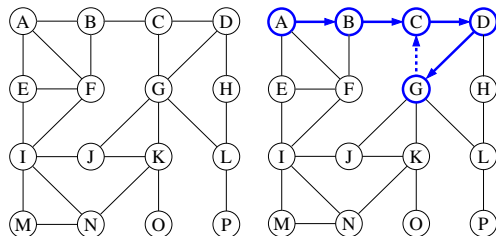
DFS starting from BOS



-----> Back edge
—————> Tree edge (discovery edge)



Depth First Search: Algorithm

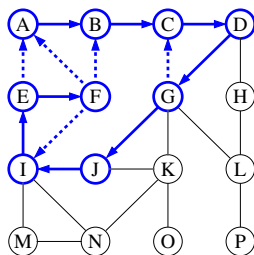


$A \rightarrow B \rightarrow C \rightarrow D \rightarrow G \rightarrow C?$

Algorithm 1: DFS(G, u)

```
1 record( $u$ );  
2 for ( $u, v$ )  $\in G$  do  
3   | if  $v$  was not visited then  
4   |   | DFS( $G, v$ );  
5   | end  
6 end
```

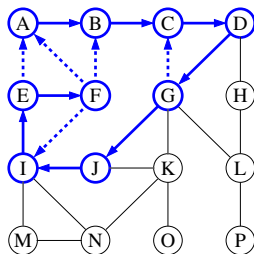
DFS



```
private static void dfs(Graph G, int v) {  
    marked[v] = true;  
    for (Edge e : G.edgesOf[v]) {  
        int adj = e.to();  
        if (!marked[adj])  
            dfs(G, adj);  
    }  
}
```

G→J

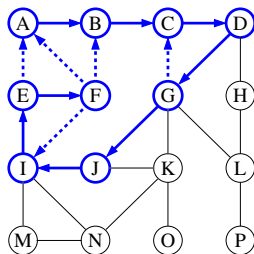
DFS



```
private static void dfs(Graph G, int v) {
    marked[v] = true;
    for (Edge e : G.edgesOf[v]) {
        int adj = e.to();
        if (!marked[adj])
            dfs(G, adj);
    }
}
```

$$G \rightarrow J \rightarrow I$$

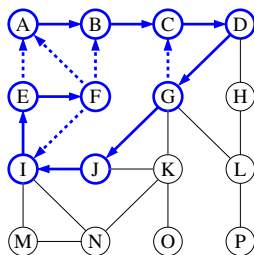
DFS



```
private static void dfs(Graph G, int v) {
    marked[v] = true;
    for (Edge e : G.edgesOf[v]) {
        int adj = e.to();
        if (!marked[adj])
            dfs(G, adj);
    }
}
```

$$G \rightarrow J \rightarrow I \rightarrow E$$

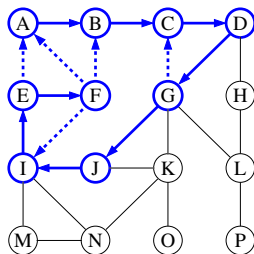
DFS



```
private static void dfs(Graph G, int v) {  
    marked[v] = true;  
    for (Edge e : G.edgesOf[v]) {  
        int adj = e.to();  
        if (!marked[adj])  
            dfs(G, adj);  
    }  
}
```

$G \rightarrow J \rightarrow I \rightarrow E \rightarrow A$?

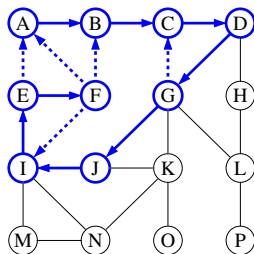
DFS



```
private static void dfs(Graph G, int v) {
    marked[v] = true;
    for (Edge e : G.edgesOf[v]) {
        int adj = e.to();
        if (!marked[adj])
            dfs(G, adj);
    }
}
```

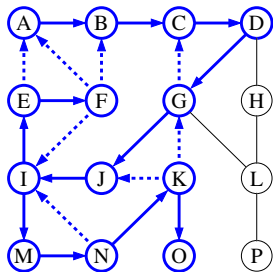
$$G \rightarrow J \rightarrow I \rightarrow E \rightarrow A? \rightarrow F$$

DFS

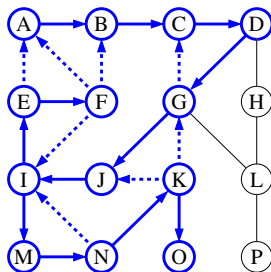


```
private static void dfs(Graph G, int v) {
    marked[v] = true;
    for (Edge e : G.edgesOf(v)) {
        int adj = e.to();
        if (!marked[adj])
            dfs(G, adj);
    }
}
```

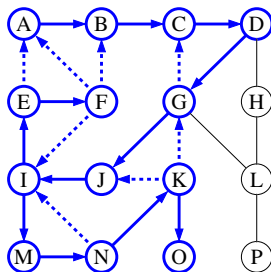
$$G \rightarrow J \rightarrow I \rightarrow E \rightarrow A? \rightarrow F \rightarrow B?$$



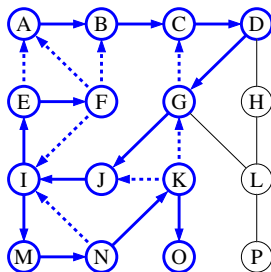
I



$I \rightarrow M$

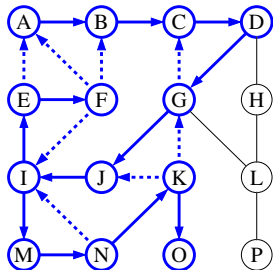


$I \rightarrow M \rightarrow N$

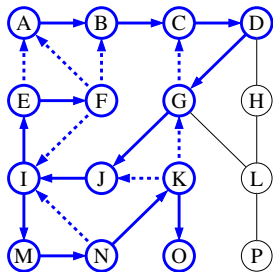


$I \rightarrow M \rightarrow N \rightarrow I ?$

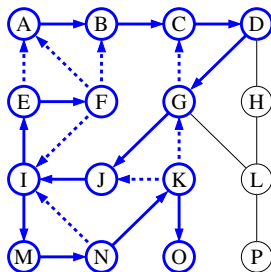
DFS


$$I \rightarrow M \rightarrow N \rightarrow I ? \rightarrow K$$

DFS

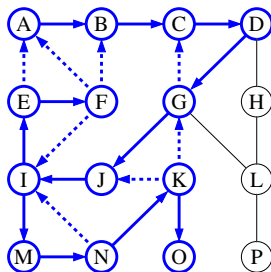

$$I \rightarrow M \rightarrow N \rightarrow I ? \rightarrow K \rightarrow J?$$

DFS



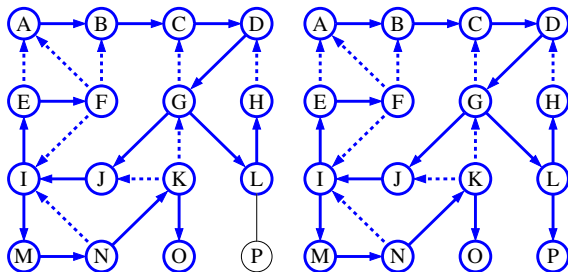
$I \rightarrow M \rightarrow N \rightarrow I ? \rightarrow K \rightarrow J ? \rightarrow G ?$

DFS



$I \rightarrow M \rightarrow N \rightarrow I ? \rightarrow K \rightarrow J ? \rightarrow G ? \rightarrow O$

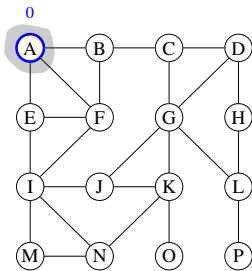
DFS



BFS (Breadth first search)

Applications

- ▶ Web crawling
- ▶ Social networks
- ▶

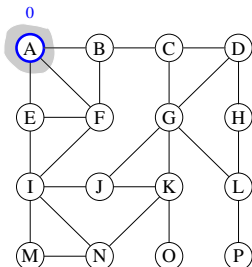


$Q=(a)$

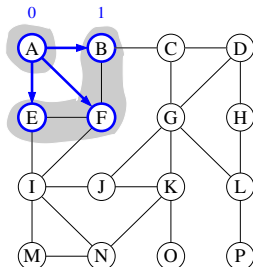
BFS (Breadth first search)

Applications

- ▶ Web crawling
- ▶ Social networks
- ▶



$Q=(a)$

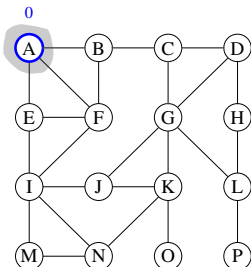


$Q=(E,F,B)$

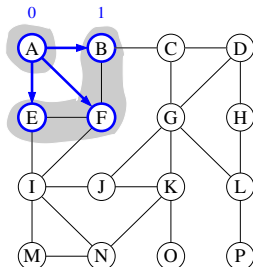
BFS (Breadth first search)

Applications

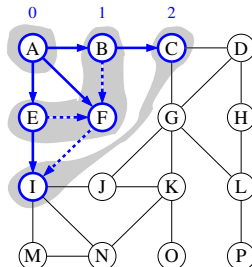
- ▶ Web crawling
- ▶ Social networks
- ▶



$Q=(a)$



$Q=(E,F,B)$



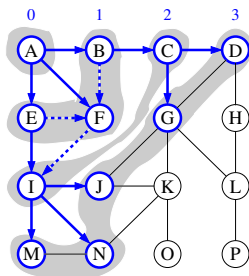
$Q=(I,C)$

Algorithm 2: BFS(G, u)

```

1 add u to queue Q.
2 while Q not empty do
3     v=Q.dequeue();
4     record(v)
5     add all unmarked neighbour of v to Q.
6     mark those neighbours
7 end

```

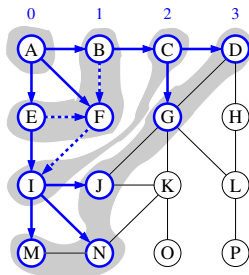
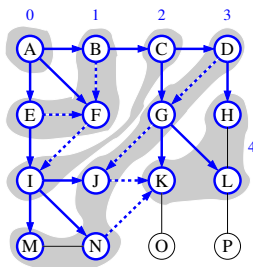

$$Q=(M,N,J,G,D)$$

Algorithm 3: BFS(G, u)

```

1 add  $u$  to queue  $Q$ .
2 while  $Q$  not empty do
3    $v = Q.dequeue()$ ;
4   record( $v$ )
5   add all unmarked neighbour of  $v$  to  $Q$ .
6   mark those neighbours
7 end

```

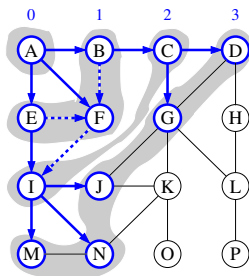
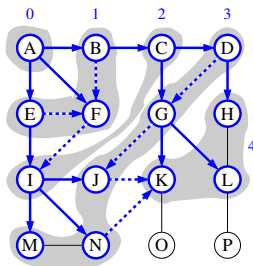
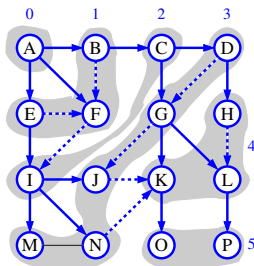
 $Q=(M,N,J,G,D)$  $Q=(K,L,H)$

Algorithm 4: BFS(G, u)

```

1 add  $u$  to queue  $Q$ .
2 while  $Q$  not empty do
3    $v = Q.dequeue()$ ;
4   record( $v$ )
5   add all unmarked neighbour of  $v$  to  $Q$ .
6   mark those neighbours
7 end

```

 $Q = (M, N, J, G, D)$  $Q = (K, L, H)$  $Q = (O, P)$

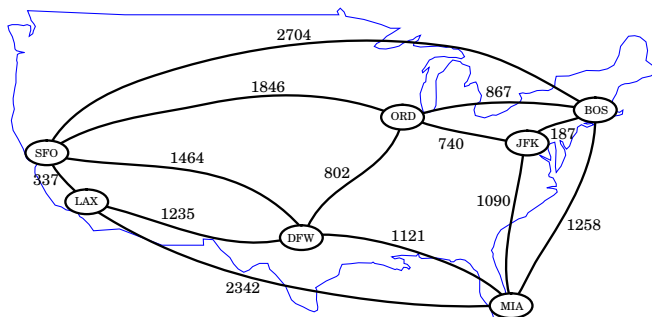
BFS Code

```
public static Queue<Integer> BFS(Graph G, int s) {
    Queue<Integer> visited = new LinkedList<Integer>();
    boolean[] marked = new boolean[G.V]; // node in queue
    int[] distTo = new int[G.V]; // distance to the source s
    int[] edgeTo = new int[G.V]; // previous edge on shortest path
    Queue<Integer> q = new LinkedList<Integer>();
    for (int v = 0; v < G.V; v++)
        distTo[v] = Integer.MAX_VALUE;
    distTo[s] = 0;
    q.add(s);
    marked[s] = true;

    while (!q.isEmpty()) {
        int v = q.remove();
        visited.add(v);
        for (Edge e : G.edgesOf[v]) {
            int w = e.to(); // w is a neighbour
            if (!marked[w]) {
                edgeTo[w] = v;
                distTo[w] = distTo[v] + 1;
                q.add(w);
                marked[w] = true;
            }
        }
    }
    return visited;
}
```

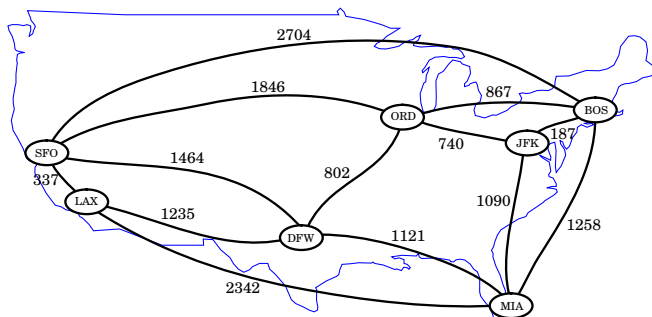
It is a shortest path algorithm for unweighted graph.

Shortest path (for weighted graph)



- ▶ Weighted graph: edges have weight
- ▶ E.g.: the weight is the distance between airports.
- ▶ What is the time complexity of a bad algorithm?

Shortest path (for weighted graph)



- ▶ Weighted graph: edges have weight
- ▶ E.g.: the weight is the distance between airports.
- ▶ What is the time complexity of a bad algorithm?
- ▶ It can be exponential

Shortest path and Dijkstra's algorithm: problem definition

There are variants of shortest path problem and Dijkstra algorithms

- ▶ Graphs
 - ▶ Weighted/unweighted graph
 - ▶ Directed/undirected graphs
 - ▶ Whether weight can be negative
- ▶ Solutions
 - ▶ (s,t) *shortest path*: find the shortest path from node s to node t .
 - ▶ *single-source shortest path*: from a source node s , find shortest paths to all other nodes.
- ▶ Output
 - ▶ The shortest Path
 - ▶ The path length only (the original algorithm)
- ▶ Implementations
 - ▶ Unsorted array
 - ▶ Hash?
 - ▶ Priority queue(Heap) ?

Greedy algorithm

- ▶ Greedy algorithm is an algorithm design paradigm
- ▶ In each stage make the local optimal choice
- ▶ Often used in optimization problems
- ▶ Dijkstra's algorithm is an example of greedy algorithms
- ▶ Compare with other paradigms
 - ▶ Divide and Conquer (e.g., merge sort, quick Sort)
 - ▶ Dynamic programming (e.g., Fibonacci, edit distance)

Dijkstra's algorithm

- ▶ Implement Dijkstra's algorithm with Priority Queue.
- ▶ Question: how to update Q efficiently?

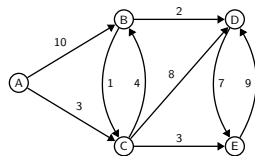
Algorithm 5: Dijkstra's algorithm

Input: Graph $G=(V, E)$, starting node s

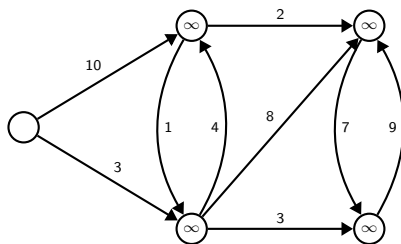
Output: Distance *dist* to all remaining nodes

```
1 dist[s] = 0;
2 dist[v] =  $\infty$  for all other nodes;
3 PriorityQueue Q = V with their distances;
4 while Q not empty do
5     u = Q.removeMin();
6     for all  $v \in neighbors[u]$  and  $v \in Q$  do
7         if  $dist[v] > dist[u] + w(u, v)$  then
8             dist[v] =  $dist[u] + w(u, v)$ ;
9             update Q with new distance
10        end
11    end
12 end
13 return dist
```

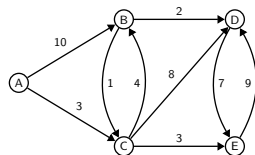
Tracing the algorithm



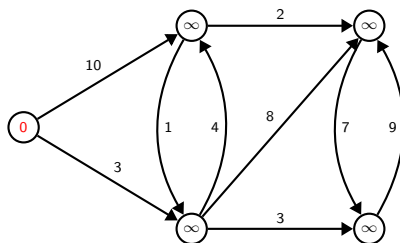
0	A	B	C	D	E	Q	u
0	0	∞	∞	∞	∞	A,B,C,D,E	A



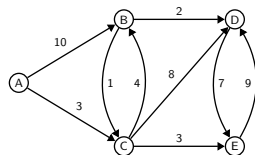
Tracing the algorithm



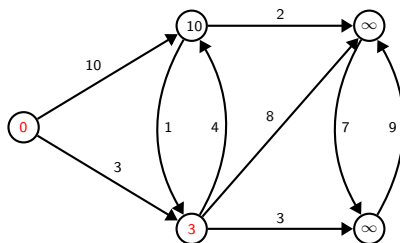
	A	B	C	D	E	Q	u
0	0	∞	∞	∞	∞	A,B,C,D,E	A
1	0	10	3	∞	∞	B,C,D,E	C



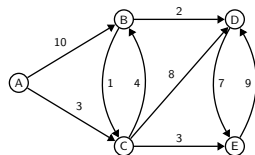
Tracing the algorithm



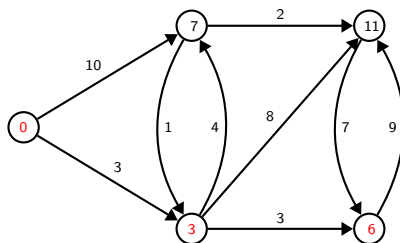
	A	B	C	D	E	Q	u
0	0	∞	∞	∞	∞	A,B,C,D,E	A
1	0	10	3	∞	∞	B,C,D,E	C
2	0	7	3	11	6	B,D,E	E



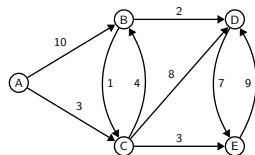
Tracing the algorithm



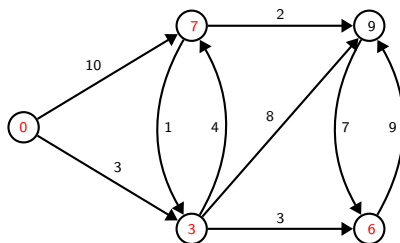
	A	B	C	D	E	Q	u
0	0	∞	∞	∞	∞	A,B,C,D,E	A
1	0	10	3	∞	∞	B,C,D,E	C
2	0	7	3	11	6	B,D,E	E
3	0	7	3	11	6	B,D	B



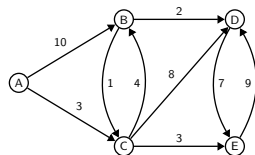
Tracing the algorithm



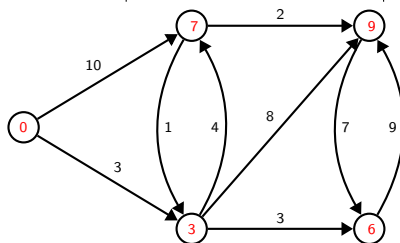
	A	B	C	D	E	Q	u
0	0	∞	∞	∞	∞	A,B,C,D,E	A
1	0	10	3	∞	∞	B,C,D,E	C
2	0	7	3	11	6	B,D,E	E
3	0	7	3	11	6	B,D	B
4	0	7	3	9	6	D	D



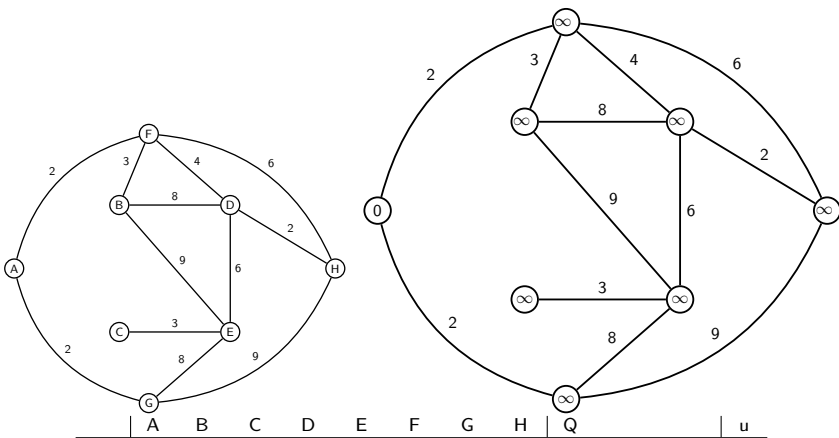
Tracing the algorithm



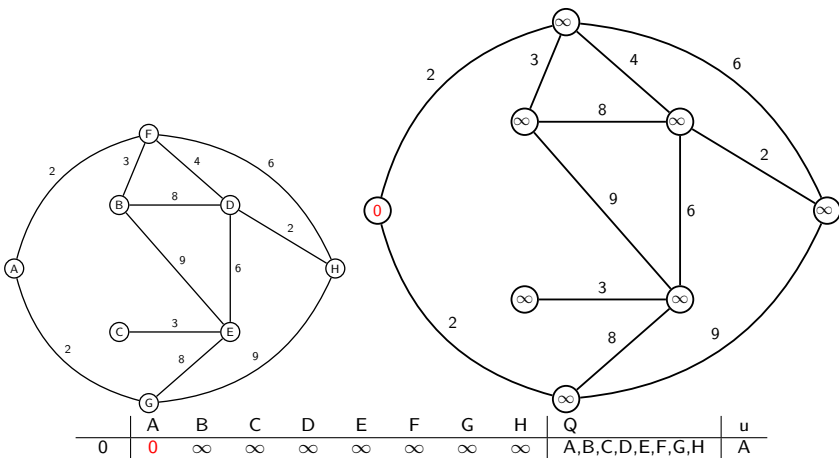
	A	B	C	D	E	Q	u
0	0	∞	∞	∞	∞	A,B,C,D,E	A
1	0	10	3	∞	∞	B,C,D,E	C
2	0	7	3	11	6	B,D,E	E
3	0	7	3	11	6	B,D	B
4	0	7	3	9	6	D	D
5	0	7	3	9	6		



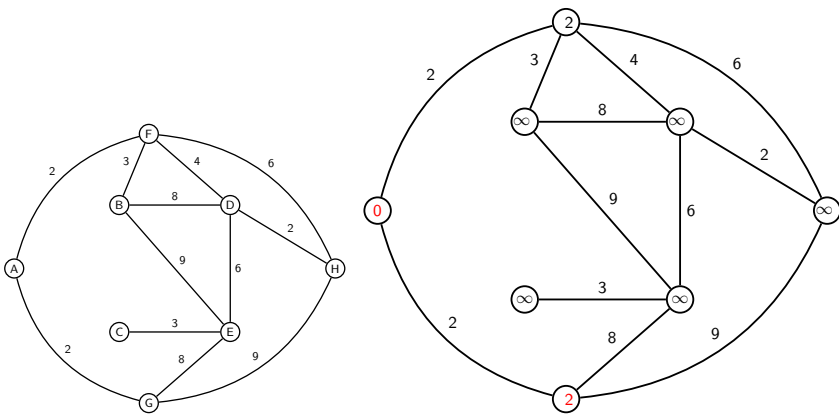
Tracing the algorithm: second example



Tracing the algorithm: second example

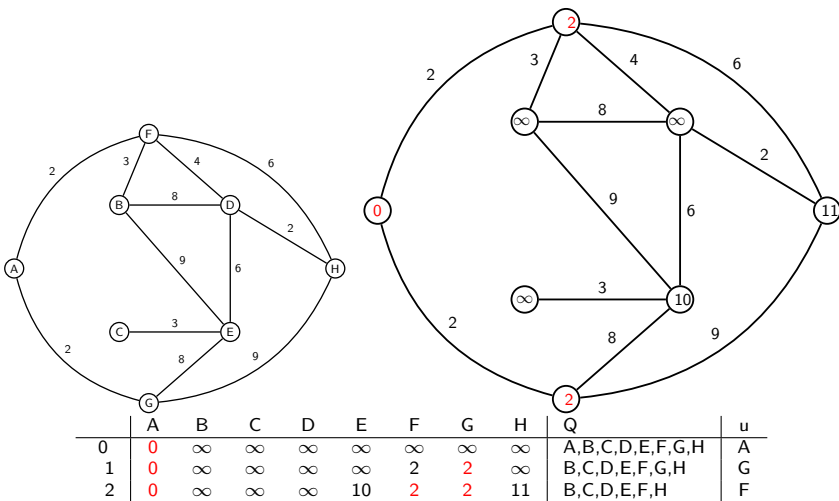


Tracing the algorithm: second example

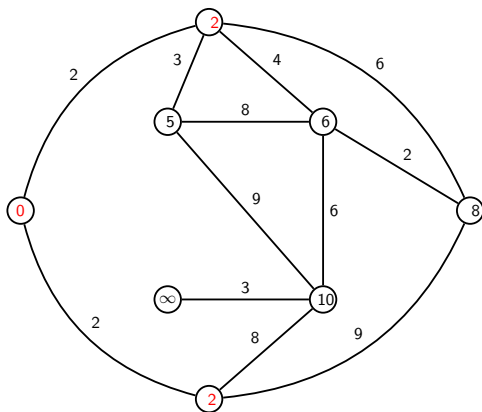
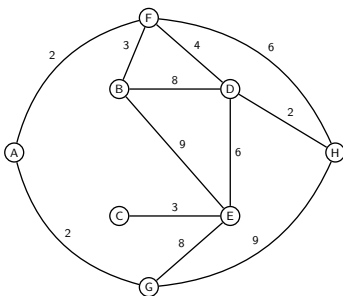


	A	B	C	D	E	F	G	H	Q	u
0	0	∞	∞	∞	∞	∞	∞	∞	A, B, C, D, E, F, G, H	A
1	0	∞	∞	∞	∞	2	2	∞	B, C, D, E, F, G, H	G

Tracing the algorithm: second example

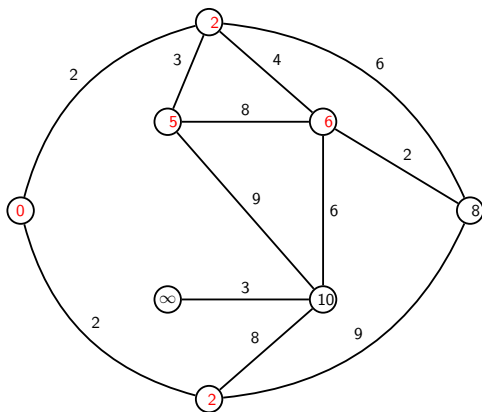
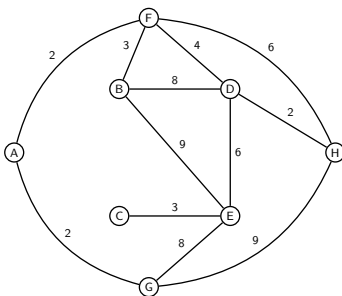


Tracing the algorithm: second example



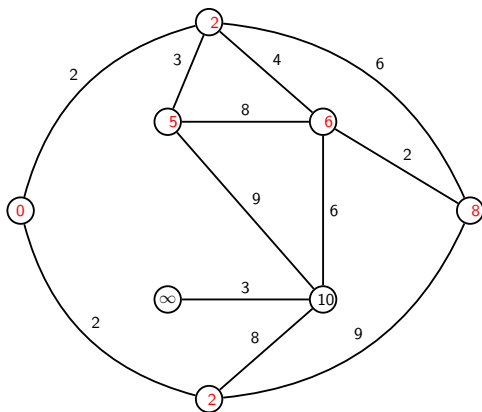
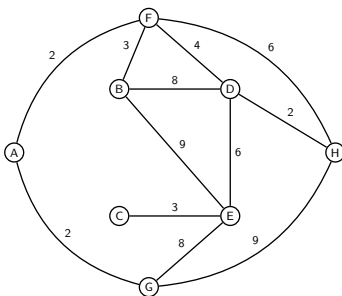
	A	B	C	D	E	F	G	H	Q	u
0	0	∞	∞	∞	∞	∞	∞	∞	A, B, C, D, E, F, G, H	A
1	0	∞	∞	∞	∞	2	2	∞	B, C, D, E, F, G, H	G
2	0	∞	∞	∞	10	2	2	11	B, C, D, E, F, H	F
3	0	5	∞	6	10	2	2	8	B, C, D, E, H	B

Tracing the algorithm: second example



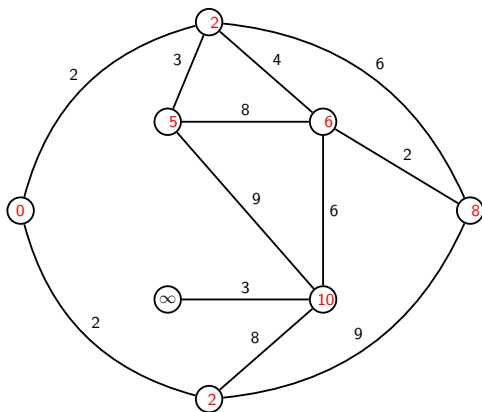
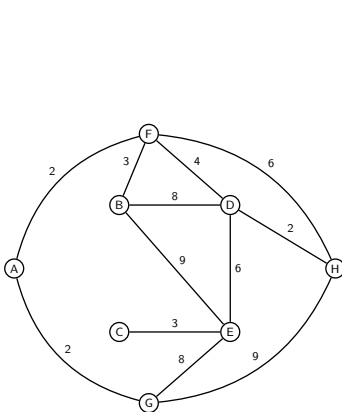
	A	B	C	D	E	F	G	H	Q	u
0	0	∞	∞	∞	∞	∞	∞	∞	A, B, C, D, E, F, G, H	A
1	0	∞	∞	∞	∞	2	2	∞	B, C, D, E, F, G, H	G
2	0	∞	∞	∞	10	2	2	11	B, C, D, E, F, H	F
3	0	5	∞	6	10	2	2	8	B, C, D, E, H	B
4	0	5	∞	6	10	2	2	8	C, D, E, H	D

Tracing the algorithm: second example



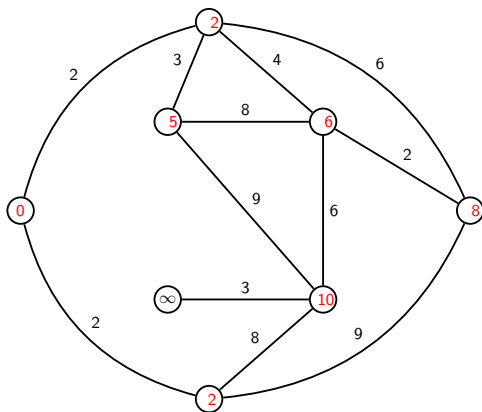
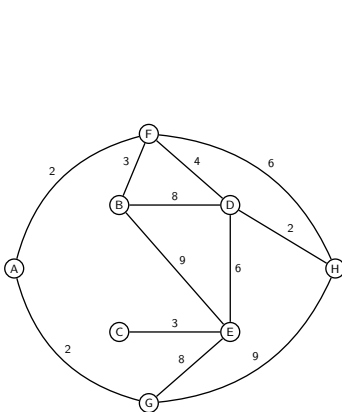
	A	B	C	D	E	F	G	H	Q	u
0	0	∞	∞	∞	∞	∞	∞	∞	A, B, C, D, E, F, G, H	A
1	0	∞	∞	∞	∞	2	2	∞	B, C, D, E, F, G, H	G
2	0	∞	∞	∞	10	2	2	11	B, C, D, E, F, H	F
3	0	5	∞	6	10	2	2	8	B, C, D, E, H	B
4	0	5	∞	6	10	2	2	8	C, D, E, H	D
5	0	5	∞	6	10	2	2	8	C, E, H	H

Tracing the algorithm: second example



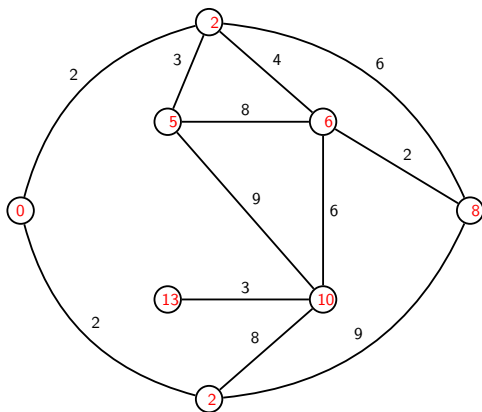
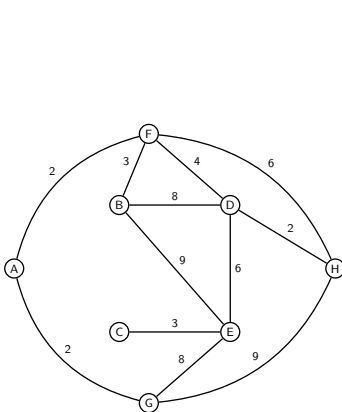
	A	B	C	D	E	F	G	H	Q	u
0	0	∞	∞	∞	∞	∞	∞	∞	A,B,C,D,E,F,G,H	A
1	0	∞	∞	∞	∞	2	2	∞	B,C,D,E,F,G,H	G
2	0	∞	∞	∞	10	2	2	11	B,C,D,E,F,H	F
3	0	5	∞	6	10	2	2	8	B,C,D,E,H	B
4	0	5	∞	6	10	2	2	8	C,D,E,H	D
5	0	5	∞	6	10	2	2	8	C,E, H	H
6	0	5	∞	6	10	2	2	8	C,E	E

Tracing the algorithm: second example



	A	B	C	D	E	F	G	H	Q	u
0	0	∞	∞	∞	∞	∞	∞	∞	A,B,C,D,E,F,G,H	A
1	0	∞	∞	∞	∞	2	2	∞	B,C,D,E,F,G,H	G
2	0	∞	∞	∞	10	2	2	11	B,C,D,E,F,H	F
3	0	5	∞	6	10	2	2	8	B,C,D,E,H	B
4	0	5	∞	6	10	2	2	8	C,D,E,H	D
5	0	5	∞	6	10	2	2	8	C,E, H	H
6	0	5	∞	6	10	2	2	8	C,E	E
7	0	5	13	6	10	2	2	8	C	C

Tracing the algorithm: second example



	A	B	C	D	E	F	G	H	Q	u
0	0	∞	∞	∞	∞	∞	∞	∞	A,B,C,D,E,F,G,H	A
1	0	∞	∞	∞	∞	2	2	∞	B,C,D,E,F,G,H	G
2	0	∞	∞	∞	10	2	2	11	B,C,D,E,F,H	F
3	0	5	∞	6	10	2	2	8	B,C,D,E,H	B
4	0	5	∞	6	10	2	2	8	C,D,E,H	D
5	0	5	∞	6	10	2	2	8	C,E, H	H
6	0	5	∞	6	10	2	2	8	C,E	E
7	0	5	13	6	10	2	2	8	C	C

Dijkstra's algorithm

Overall idea:

1. Base case: Start with $\text{Explored} = s$.

2. Inductive step:

Optimal substructure: After having computed the shortest path to all vertices in Explored ,

Greedy choice: extend Explored with a v that can be reached using one edge e from some $u \in \text{Explored}$ such that

$$\text{dist}(u) + w(e) \text{ is minimized} \quad (3)$$

3. Finish: when $\text{Explored} = V$

Complexity of Dijkstra's algorithm (the good implementation)

- ▶ Suppose that n is number of nodes, m is number of edges
- ▶ n insertions into Q .
- ▶ n calls to the removeMin method on Q .
 - ▶ removeMin is $\log n$. Hence this part is $n \log n$
- ▶ m calls to the distance checking and updating
 - ▶ Updating Adaptable priority queue is $O(\log n)$
- ▶ The complexity is $O((n + m) \log n)$
- ▶ It is also correct to say that it is $O(m \log n)$ because $n < m$.
- ▶ There are other variants of implementation that have worse time complexity.

Run Shortest Path Algorithm

- ▶ construct a graph
- ▶ find the shortest paths from s
- ▶ print some paths and distances

```
public class Graph_client {
    public static void main(String[] args) throws Exception {
        //construct a graph from a text file
        Graph G = new Graph(new Scanner(new File("graph1.txt")));
        int s = 0;
        //find all the shortest paths of G starting from node s
        //saves both distances (in distTo) and paths (in edgeTo)
        Graph_SP sp = new Graph_SP(G, s);
        // print the first a few shortest paths
        for (int t = 0; t < 5; t++) {
            if (sp.distTo[t] < Double.POSITIVE_INFINITY) {
                System.out.printf("%d => %d (%.2f) ", s, t, sp.distTo[t]);
                String path = t + "";
                //print the path to node t
                for (Edge e = sp.edgeTo[t]; e != null; e = sp.edgeTo[e.from()]) {
                    path = e.from() + "->(" + e.weight() + ")>" + path;
                }
                System.out.println(path);
            } else
                System.out.printf("no path");
        }
    }
}
```

The dilemma of the Dijkstra's algorithm

Need to implement two operations efficiently:

- ▶ removeMin (use Priority Queue)
- ▶ Find and Update (use HashTable)

Algorithm 6: Dijkstra's algorithm

Input: Graph $G=(V, E)$, starting node s

Output: Distance $dist$ to all remaining nodes

```
1 dist[s] = 0;
2 dist[v] =  $\infty$  for all other nodes;
3 PriorityQueue Q = V with their distances;
4 while Q not empty do
5     u = Q.removeMin();
6     for all  $v \in neighbors[u]$  and  $v \in Q$  do
7         if  $dist[v] > dist[u] + w(u, v)$  then
8              $dist[v] = dist[u] + w(u, v)$ ;
9             update Q with new distance (put(v, dist(v)))
10        end
11    end
12 end
13 return dist
```


First round: Shortest Path With a Bad (but easier) Data Structure

```
public class Graph_SP {
    public double[] distTo; //record the distances
    public Edge[] edgeTo; //records the edges along the paths
    public Map distTable=new HashMap<Integer , Double>(); //track
        remaining nodes
    public Graph_SP(Graph G, int s) {
        distTo = new double[G.V]; //array of length of number of nodes
        edgeTo = new Edge[G.V];
        //Initially every node has infinitely large distance
        for (int v = 0; v < G.V; v++)
            distTo[v] = Double.POSITIVE_INFINITY;
        distTo[s] = 0.0; // starting node has zero distance

        // relax vertices in order of distance from s
        distTable.put(s, distTo[s]);
        while (!distTable.isEmpty()) {
            int v = removeMin(distTable);
            for (Edge e : G.edgesOf[v])
                relax(e);
        }
    }
}
```

Relaxation

```
private void relax(Edge e) {  
    int v = e.from(), w = e.to();  
    if (distTo[w] > distTo[v] + e.weight()) {  
        distTo[w] = distTo[v] + e.weight();  
        edgeTo[w] = e;  
        distTable.put(w, distTo[w]);  
    }  
}
```

removeMin is slow (in bad implementation)

```
public static int removeMin(Map<Integer, Double> distTable) {
    Iterator it = distTable.entrySet().iterator();
    double minValue = Integer.MAX_VALUE;
    int minKey = -1;
    while (it.hasNext()) {
        Map.Entry pair = (Map.Entry) it.next();

        if (minValue > (Double) pair.getValue()) {
            minKey = (Integer) pair.getKey();
            minValue = (Double) pair.getValue();
        }
    }
    distTable.remove(minKey);
    return minKey;
}
```

Time complexity for the naive implementation:

- ▶ m edges need to be checked/updated.
 - ▶ Check is fast for HashMap ($O(1)$).
- ▶ There are n removeMin operations.
 - ▶ the cost of RemoveMin is $O(n)$
- ▶ Hence altogether it is $O(m) + O(n^2)$
- ▶ It can be simplified as $O(n^2)$ because $m < n^2$.

The Better One: Heap2540_SP

It is similar to Heap2540, but adapted for the SP problem

- ▶ minHeap, not maxHeap. (change less(..) to greater(..))
- ▶ need to save both the node and the distance (use distances)
- ▶ need to locate and update quickly (use index)

```
public class Heap2540_SP {
    int[] heap;
    int[] index;
    Double[] distances;
    int n = 0; // actual heap size.
    int CAPACITY = 1000001; // array size

    public Heap2540_SP() {
        heap = new int[CAPACITY];
        index = new int[CAPACITY];
        distances = new Double[CAPACITY];
        for (int i = 0; i < CAPACITY; i++)
            index[i] = -1;
    }
}
```

The Better One: Heap2540_SP

```
public int removeMin()
    int min = heap[1];
    swap(1, n);
    n--;
    sink(1);
    index[min] = -1;
    distances[min] = null;
    return min;

public void insert(int node, double dist)
    n++;
    heap[n] = node;
    index[node] = n;
    distances[node] = dist;
    swim(n);

private void swim(int k)
    while (k > 1 && greater(k / 2, k)) {
        swap(k, k / 2);
        k = k / 2;
    }

public void put(int node, double dist)
    if (index[node] != -1) {
        distances[node] = dist;
        swim(index[node]);
    } else
        insert(node, dist);
```

The Better One: Heap2540_SP

```
private void swap(int i, int j) {
    Integer temp = heap[i];
    heap[i] = heap[j];
    heap[j] = temp;
    // add missing code here
}

private void sink(int k) {
    while (2 * k <= n) {
        int j = 2 * k;
        if (j < n && greater(j, j + 1))
            j++;
        if (!greater(k, j))
            break;
        swap(k, j);
        k = j;
    }
}

// check whether the distance of the node in heap[i] is greater
// than the distance of node in heap[j]
private boolean greater(int i, int j) {
    //heap[i]>heap[j] is not the correct answer. it compares the node
    //IDs, which is meaningless. We need to compare their distances.
}
}
```

Lessons learned

- ▶ Four levels of programmers
 1. Can write the code (word counts with $O(nmm)$ time complexity)
 2. Can use API correctly (avoid using `get(i)`, obtain $O(nm)$ time complexity)
 3. Can select better data structure (Use hashtable)
 4. Can modify the API (modify the heap in Dijkstra algorithm)
- ▶ Why not justing making PQ adaptable?
 - ▶ Extra cost (need the index array)
 - ▶ Normally not needed

Takeaways

- ▶ Graph definition, $G=(V,E)$. Basic properties. Path. SCC, WCC.
- ▶ Graph representations. Adjacency Matrix, Edge list, Adjacency List.
- ▶ Graph traversal. DFS. BFS.
- ▶ Shortest path. Dijkstra's algorithm. Greedy algorithm.
- ▶ Data structures for the Dijkstra's algorithm
- ▶ Three steps to understand Dijkstra algorithm:
 - ▶ BFS: Shortest path algorithm for unweight graph
 - ▶ GraphSP: shortest path without heap
 - ▶ Dijkstra algorithm with adaptable heap/priority queue
- ▶ Readings:
 - ▶ P611-642. P 653-661. Goodrich et al.
 - ▶ [Algorithms, 4th Edition, By Sedgewick et al.](#) (clickable).